

Partie 1 : Les Fondamentaux

Chapitre 1 : Syntaxe, Variables et Types de Données

Python est un langage à typage dynamique et fort.

- **Variables & Affectation** : Pas besoin de déclarer le type.
- **Types primitifs** : `int`, `float`, `str`, `bool`, `NoneType`.
- **Propreté du code** : Respect de la norme **PEP 8** (indentation de 4 espaces, noms en `snake_case`).

```
# Typage dynamique
age = 25 # int
taille = 1.75 # float
nom = "Alice" # str
est_actif = True # bool

# F-strings (formatage moderne)
message = f"{nom} a {age} ans."
```

Chapitre 2 : Structures de Contrôle et Fonctions

Le flux d'exécution repose sur les conditions, les boucles et la modularité via les fonctions.

- **Conditions** : `if`, `elif`, `else`.
- **Boucles** : `for` (itération sur des itérables), `while` (condition tant que).
- **Fonctions** : Définition avec `def`, arguments positionnels, arguments par défaut, `*args` et `**kwargs`.

```
def calculer_remise(prix: float, remise: float = 0.1, *tags) -> float:
    """Calcule le prix après remise et applique les règles selon les
    tags."""
    prix_final = prix * (1 - remise)
    if "VIP" in tags:
        prix_final -= 5
    return max(prix_final, 0.0)

resultat = calculer_remise(100, 0.2, "VIP")
```

Chapitre 3 : Structures de Données Intégrées

Les quatre conteneurs natifs majeurs :

- **Listes** (`list`) : Ordonnées, mutables (`[1, 2, 3]`).

- **Tuples (`tuple`)** : Ordonnés, immutables (`(1, 2, 3)`).
- **Dictionnaires (`dict`)** : Clé-valeur, recherche en $\mathcal{O}(1)$ (`{"cle": "valeur"}`).
- **Ensembles (`set`)** : Non ordonnés, éléments uniques (`{1, 2, 3}`).

```
# Comprehensions de listes et dictionnaires
carres_pairs = [x**2 for x in range(10) if x % 2 == 0]
map_carres = {x: x**2 for x in range(5)}
```

Partie 2 : Programmation Orientée Objet (POO)

Chapitre 4 : Classes, Objets et Encapsulation

La POO permet de modéliser des entités complexes.

- **Attributs & Méthodes** : `__init__` (constructeur), `self`.
- **Visibilité** : Encapsulation par convention (`_protege`, `__prive`).
- **Propriétés** : Décorateur `@property` pour les getters/setters pythoniques.

```
class CompteBancaire:

    def __init__(self, titulaire: str, solde_initial: float = 0.0):
        self.titulaire = titulaire
        self.__solde = solde_initial # Attribut privé

    @property
    def solde(self) -> float:
        return self.__solde

    def deposer(self, montant: float) -> None:
        if montant > 0:
            self.__solde += montant

compte = CompteBancaire("Jean", 100.0)
compte.deposer(50.0)
print(compte.solde) # 150.0
```

Chapitre 5 : Héritage, Polymorphisme et Méthodes Magiques

- **Héritage** : Réutilisation avec `super()`.
- **Méthodes Spéciales (*Dunder Methods*)** : `__str__`, `__repr__`, `__eq__`, `__len__`, `__getitem__`.

```
class Employe:

    def __init__(self, nom: str, salaire: float):
        self.nom = nom
        self.salaire = salaire

    def __repr__(self):
        return f"Employe(nom='{self.nom}', salaire={self.salaire})"

class Developpeur(Employe):

    def __init__(self, nom: str, salaire: float, langage: str):
        super().__init__(nom, salaire)
        self.langage = langage
```

Partie 3 : Python Avancé & Interne du Langage

Chapitre 6 : Décorateurs et Introspection

Un décorateur est une fonction qui prend une autre fonction en argument pour en modifier le comportement sans altérer son code source.

```
import functools
import time

def mesurer_temps(func):

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        debut = time.perf_counter()
        resultat = func(*args, **kwargs)
        duree = time.perf_counter() - debut
        print(f"[{func.__name__}] Exécuté en {duree:.6f}s")
        return resultat

    return wrapper

@mesurer_temps
def traitement_lourd():
    return sum(i**2 for i in range(1_000_000))

traitement_lourd()
```

Chapitre 7 : Itérateurs, Générateurs et Protocoles

Les générateurs produisent des valeurs à la volée (*lazy evaluation*) via le mot-clé `yield`, permettant d'économiser la mémoire sur de grands volumes de données.

```
def flux_donnees(max_items: int):
    valeur = 0
    while valeur < max_items:
        yield valeur
        valeur += 1

# Consommation mémoire minimale
for item in flux_donnees(1_000_000):
    if item > 5:
        break
    print(item)
```

Chapitre 8 : Gestionnaires de Contexte (*Context Managers*)

Garantissent la libération des ressources (fichiers, connexions réseau, verrous) via le bloc `with`.

```
# Via la classe (protocole __enter__ / __exit__)
class FichierLog:

    def __init__(self, filename: str):
        self.filename = filename

    def __enter__(self):
        self.file = open(self.filename, "a")
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()

# Via contextlib
from contextlib import contextmanager

@contextmanager
def verrou_temp():
    print("Verrou acquis")
    yield
    print("Verrou libéré")
```

```
with verrou_temp():  
    print("Opération critique")
```

Chapitre 9 : Typage Avancé, Metaclasses et Modèle Objet

- **Type Hinting Avancé** : `typing` (`Generic`, `Protocol`, `Union`, `Callable`).
- **Métaclasses** : Les classes des classes. Elles permettent d'intercepter et modifier la création des classes elles-mêmes (`type`).

```
from typing import Protocol  
  
# Structural Subtyping (Duck Typing statique)  
class ContratRenderable(Protocol):  
  
    def render(self) -> str:  
        ...  
  
class HTMLReport:  
  
    def render(self) -> str:  
        return "<h1>Rapport</h1>"  
  
def afficher(composant: ContratRenderable):  
    print(composant.render())  
  
afficher(HTMLReport()) # Valide sans héritage explicite
```

Chapitre 10 : Concurrency, Asynchronisme et Performances

- **GIL (Global Interpreter Lock)** : Impacte le multithreading sur CPU-bound tasks.
- **threading vs multiprocessing** : Multithreading pour les I/O-bound, Multiprocessing pour les CPU-bound.
- **asyncio** : Concurrency basée sur une boucle d'événements (*event loop*) avec `async` et `await`.

```
import asyncio  
  
async def fetch_data(id_req: int, delai: int):  
    await asyncio.sleep(delai)
```

```
    return f"Données {id_req}"

async def main():
    taches = [fetch_data(1, 2), fetch_data(2, 1), fetch_data(3, 3)]
    resultats = await asyncio.gather(*taches)
    print(resultats)

# Lancement du programme asynchrone
asyncio.run(main())
```
